

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

MID SEMESTER EXAMINATION

SUMMER SEMESTER, 2024–2025

DURATION: 2 HOURS

FULL MARKS: 120

SWE 4201: Object Oriented Concepts I

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer **all 4 (four)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

-
1. a) Explain the difference between an instance variable and a local variable, covering their scope and lifetime. Then write a short Java code snippet that clearly shows both in the same class. 6
(CO1)
(PO1)
 - b) Write a Java class called `Rectangle` that has `length` and `width` as private instance variables. Include a parameterized constructor, proper getters and setters, a method `area()` that returns the area. Also write a brief `main()` method that creates one `Rectangle` object and prints its area. 8
(CO3)
(PO3)
 - c) Explain what a default constructor is and what happens if you define a parameterized constructor in a class without also explicitly defining a default one. Support your answer with a small Java example that demonstrates the compile-time consequence. 8
(CO1)
(PO1)
 - d) Consider the following statements about OOP. For each one, state whether it is true or false and write one sentence justifying your answer. 4 × 2
(CO1)
(PO1)
 - i. A subclass inherits the private fields of its superclass and can access them directly.
 - ii. Method overloading and method overriding are the same concept — both allow a class to have multiple methods with the same name.
 - iii. Making all fields of a class public improves encapsulation because it simplifies access to data.
 - iv. A constructor must always have the same name as the class it belongs to.
 2. You are building a simple fantasy combat game called `Battle Arena`. Every fighter has a name, health points (HP), and attack power. A fighter can attack another fighter, reducing the target's HP by its attack power. HP can never become negative. A fighter is considered alive if its HP is greater than zero.
 A `Hero` is a special type of `Fighter`. After every attack, a hero earns 10 gold coins. In addition, every third attack made by a hero is automatically a `Power Strike`, which deals twice the hero's normal attack power instead of a normal attack.
 - a) Write the `Fighter` class based on the requirements above. Your implementation should include appropriate instance variables, constructor(s), methods, getters and setters needed to support the described behavior. 10
(CO3)
(PO3)
 - b) Using `Fighter` as the base class, write the `Hero` class. Include both a parameterized constructor and a non-parameterized constructor that creates a hero named "Warrior" with 100 health points and 15 attack power. The non-parameterized constructor must delegate to the parameterized constructor using `this()`. 12
(CO3)
(PO3)
 - c) Suppose the game is later expanded to include a new class called `Mage`. Like a `Hero`, a `Mage` is a type of `Fighter`. Unlike a hero, every fourth attack is a magic spell that deals three times the mage's normal attack power. Instead of collecting gold coins, a mage collects magic crystals after each attack. 8
(CO2)
(PO2)
 Identify which behaviors belong in `Fighter` and which belong in subclasses, including how the special attack rules for `Hero` and `Mage` should be implemented.

3. a) Explain the purpose of abstract classes and describe a situation where they are useful. 8
(CO1)
(PO1)

- b) Consider the `main()` method shown in Code Snippet 1. 14
(CO3)
(PO3)

```

1 public class Main {
2     public static void main(String[] args) {
3         Movie m1 = new Movie("Inception", "Christopher Nolan", 2010);
4
5         Movie m2 = new Movie("Interstellar");
6
7         m1.setRating(9);
8         m2.setRating(7);
9
10        System.out.println(Movie.getMovieCount());
11        System.out.println(m1.getSummary());
12        System.out.println(m2.getSummary());
13
14        System.out.println("Inception newer? " + m1.isNewerThan(m2));
15    }
16 }

```

Code Snippet 1: `main()` for Question 3.b

The requirements of the `Movie` class are -

- `getSummary()` should return a descriptive string about the movie.
- `isNewerThan(Movie other)` should compare release years and return a boolean result.
- Since `m2` is created without a release year, choose an appropriate default value and clearly state your assumption.
- Keep track of the total number of `Movie` objects created and provide access through `getMovieCount()`.
- Ratings must be between 0 and 10. Implement a helper method for validation and use it from the setter. When an invalid rating is provided, store the nearest valid value.

Using the code above and the stated requirements, write the `Movie` class. Infer any necessary fields, constructors, and methods from how the class is used.

- c) Answer the following questions about method overloading. 3 +
3 + 2
(CO1)
(PO1)
- i. Explain what method overloading is. State two requirements that must be satisfied for two methods to be considered overloaded.
 - ii. A student claims that changing only the return type of a method is sufficient to create an overloaded method. Do you agree or disagree? Justify your answer.
 - iii. Explain ambiguity in method overloading. Briefly describe a situation where an overloaded method call may become ambiguous.
4. a) “Encapsulation just means making variables private — it adds no real benefit, it only makes code harder to write.” Do you agree with this statement? Justify your answer with at least two reasons. 6
(CO2)
(PO2)

- b) Consider the program shown in Code Snippet 2 and answer the questions that follow. 4 + 4 +
4 + 2
(CO2)
(PO2)

```

1 class Account {
2

```

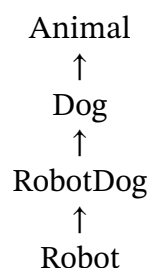
```

3      public double fee() {
4          return 100;
5      }
6  }
7
8  class PremiumAccount extends Account {
9
10     @Override
11     public double fee() {
12         return 50;
13     }
14 }
15
16 public class Main {
17
18     public static void main(String[] args) {
19
20         Account[] accounts = {
21             new Account(),
22             new PremiumAccount(),
23             new PremiumAccount()
24         };
25
26         double total = 0;
27
28         for(Account a : accounts) {
29             total += a.fee();
30         }
31
32         System.out.println(total);
33     }
34 }

```

Code Snippet 2: Program for Question 4.b

- i. Write the exact output of the program.
 - ii. Explain how the correct `fee()` method is chosen for each object. Name the OOP concept involved.
 - iii. Suppose `fee()` were declared `static` in both classes. Would the output change? Explain your answer.
 - iv. What advantage does using an `Account[]` array provide compared to using an array of `PremiumAccount` objects?
- c) A student proposes the following hierarchy:



10
(CO2)
(PO2)

Critically evaluate this inheritance hierarchy. If you believe it can be improved, explain how you would redesign it and justify your changes.